Centro Universitário de Excelência
Sistemas de Informação

Dendê Eventos: Refatoração

Autor: Anna Beatriz Silva Lima
Fátima Pereira Santos Pinho
Ian Salomão da Silva Carneiro
Rebeca Helen Batista Amorim
Valeria Soares Santos

Feira de Santana, 2026

Agenda

Apresentar a refatoração da solução Dendê Eventos, demonstrando a aplicação dos padrões de projeto Builder, DTOs, Mappers e Controllers para promover uma melhor separação de camadas, organização e manutenibilidade do código.

1. Contexto da Refatoração
Evolução da solução com aplicação de padrões de projeto

2. Diagrama de Classes
Estrutura atualizada com relacionamentos, atributos e o EventoBuilder

3. Diagrama de Atividade
Fluxo completo da User Story Alterar Evento implementada

4. Padrões de Projeto
Builder, DTOs, Mappers e Controllers ilustrados na solução

Contexto da Refatoração

A solução

Backend Java para gestão de eventos desenvolvido na primeira entrega

O desafio

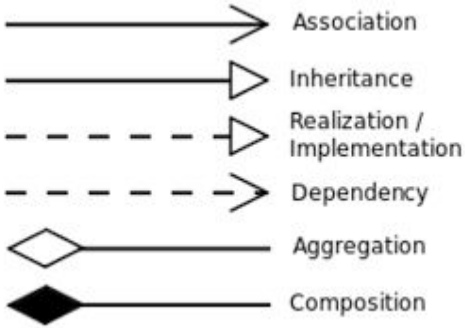
Evoluir o código tornando-o mais modular e aberto a extensões

O complemento

Criação do EventoBuilder, DTOs de request e response, Mappers e Diagrama de Atividade

Diagrama de Classes

- Melhorias feitas:
  - Adição do EventoBuilder ao diagrama
  - Relacionamentos: composição e associação corrigidos
  - Notações UML padronizadas



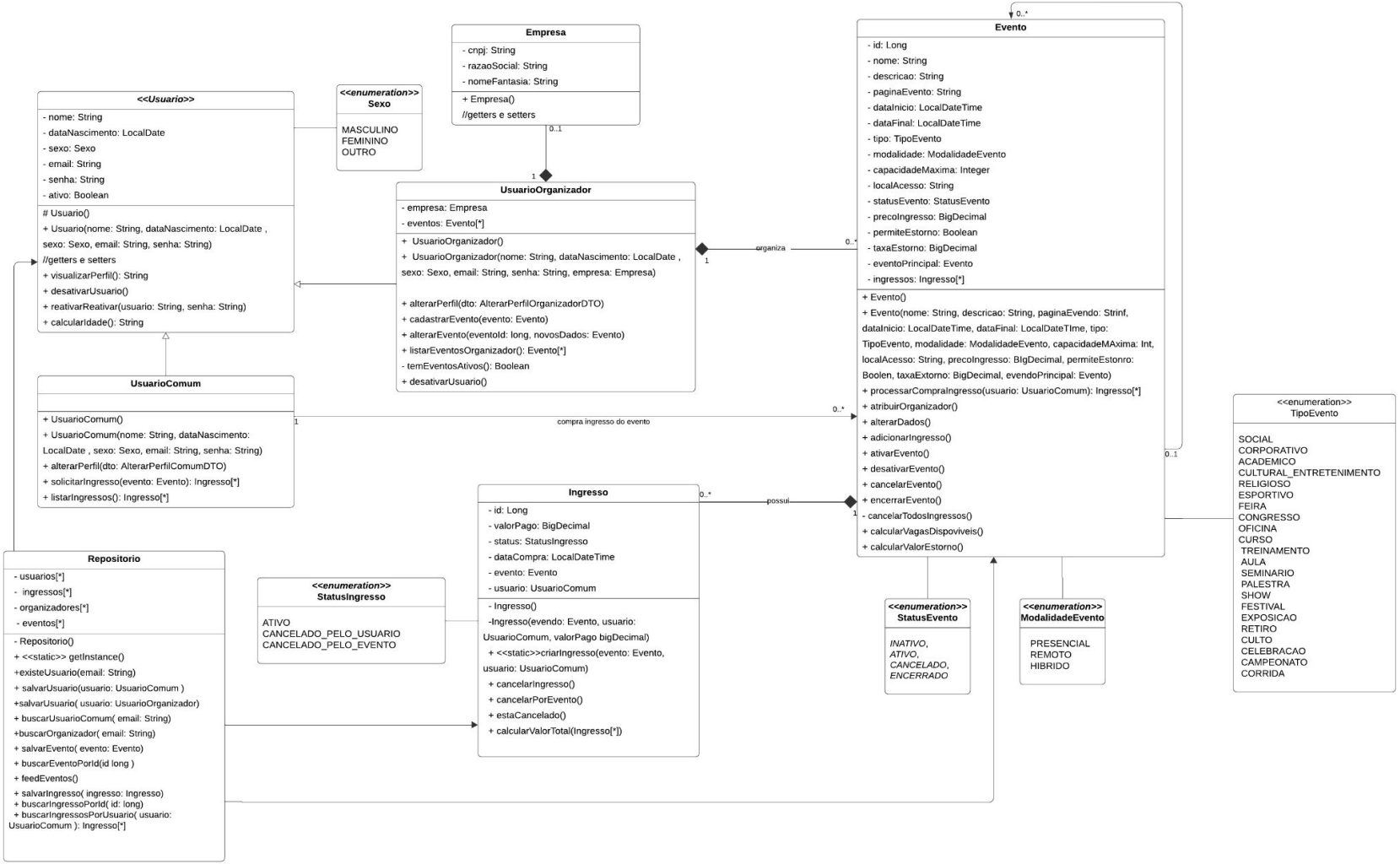
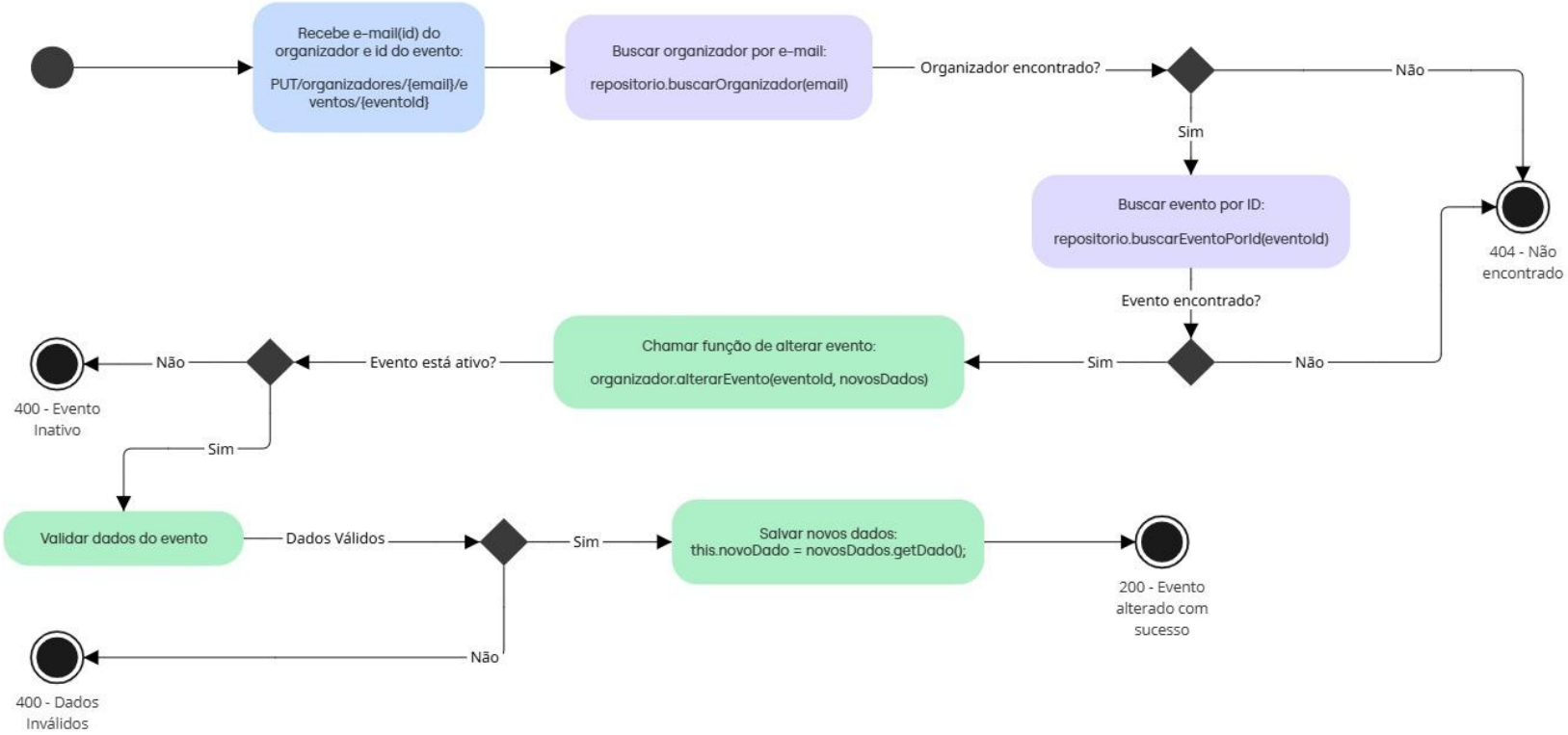

Empresa
- cnpj: String
- razaoSocial: String
- nomeFantasia: String
+ Empresa()
//getters e setters
0..1
1
<<Usuario>>
- nome: String
- dataNascimento: LocalDate
- sexo: Sexo
- email: String
- senha: String
- ativo: Boolean
Usuario()
+ Usuario(nome: String, dataNascimento: LocalDate , sexo: Sexo, email: String, senha: String)
//getters e setters
+ visualizarPerfil(): String
+ desativarUsuario()
+ reativarReativar(usuario: String, senha: String)
+ calcularIdade(): String
<<enumeration>>
Sexo
MASCULINO
FEMININO
OUTRO
UsuarioOrganizador
- empresa: Empresa
- eventos: Evento[*]
+ UsuarioOrganizador()
+ UsuarioOrganizador(nome: String, dataNascimento: LocalDate , sexo: Sexo, email: String, senha: String, empresa: Empresa)
+ alterarPerfil(dto: AlterarPerfilOrganizadorDTO)
+ cadastrarEvento(evento: Evento)
+ alterarEvento(eventoId: long, novosDados: Evento)
+ listarEventosOrganizador(): Evento[*]
- temEventosAtivos(): Boolean
+ desativarUsuario()
organiza
1
0..*
Evento
- id: Long
- nome: String
- descricao: String
- paginaEvento: String
- dataInicio: LocalDateTime
- dataFinal: LocalDateTime
- tipo: TipoEvento
- modalidade: ModalidadeEvento
- capacidadeMaxima: Integer
- localAcesso: String
- statusEvento: StatusEvento
- precoIngresso: BigDecimal
- permiteEstorno: Boolean
- taxaEstorno: BigDecimal
- eventoPrincipal: Evento
- ingressos: Ingresso[*]
+ Evento()
+ Evento(nome: String, descricao: String, paginaEvendo: Strinf, dataInicio: LocalDateTime, dataFinal: LocalDateTIme, tipo: TipoEvento, modalidade: ModalidadeEvento, capacidadeMAxima: Int, localAcesso: String, precoIngresso: BIgDecimal, permiteEstonro: Boolen, taxaExtorno: BigDecimal, evendoPrincipal: Evento)
+ processarCompraIngresso(usuario: UsuarioComum): Ingresso[*]
+ atribuirOrganizador()
+ alterarDados()
+ adicionarIngresso()
+ ativarEvento()
+ desativarEvento()
+ cancelarEvento()
+ encerrarEvento()
- cancelarTodosIngressos()
+ calcularVagasDispoviveis()
+ calcularValorEstorno()
0..*
0..1
UsuarioComum
+ UsuarioComum()
+ UsuarioComum(nome: String, dataNascimento: LocalDate , sexo: Sexo, email: String, senha: String)
+ alterarPerfil(dto: AlterarPerfilComumDTO)
+ solicitarIngresso(evento: Evento): Ingresso[*]
+ listarIngressos(): Ingresso[*]
1
compra ingresso do evento
0..*
<<enumeration>>
TipoEvento
SOCIAL
CORPORATIVO
ACADEMICO
CULTURAL_ENTRETENIMENTO
RELIGIOSO
ESPORTIVO
FEIRA
CONGRESSO
OFICINA
CURSO
TREINAMENTO
AULA
SEMINARIO
PALESTRA
SHOW
FESTIVAL
EXPOSICAO
RETIRO
CULTO
CELEBRACAO
CAMPEONATO
CORRIDA
Ingresso
- id: Long
- valorPago: BigDecimal
- status: StatusIngresso
- dataCompra: LocalDateTime
- evento: Evento
- usuario: UsuarioComum
- Ingresso()
-Ingresso(evendo: Evento, usuario: UsuarioComum, valorPago bigDecimal)
+ <<static>>criarIngresso(evento: Evento, usuario: UsuarioComum)
+ cancelarIngresso()
+ cancelarPorEvento()
+ estaCancelado()
+ calcularValorTotal(Ingresso[*])
0..*
possui
1
<<enumeration>>
StatusIngresso
ATIVO
CANCELADO_PELO_USUARIO
CANCELADO_PELO_EVENTO
<<enumeration>>
StatusEvento
INATIVO,
ATIVO,
CANCELADO,
ENCERRADO
<<enumeration>>
ModalidadeEvento
PRESENCIAL
REMOTO
HIBRIDO
Repositorio
- usuarios[*]
- ingressos[*]
- organizadores[*]
- eventos[*]
- Repositorio()
+ <<static>> getInstance()
+existeUsuario(email: String)
+ salvarUsuario(usuario: UsuarioComum)
+salvarUsuario(usuario: UsuarioOrganizador)
+ buscarUsuarioComum(email: String)
+buscarOrganizador(email: String)
+ salvarEvento(evento: Evento)
+ buscarEventoPorId(id long)
+ feedEventos()
+ salvarIngresso(ingresso: Ingresso)
+ buscarIngressoPorId(id: long)
+ buscarIngressosPorUsuario(usuario: UsuarioComum): Ingresso[*]

Diagrama de Atividade



Padrões de Projeto

unex

Builder - Padrão Criacional

Constrói objetos complexos passo a passo, evitando construtores com muitos parâmetros

DTOs - Padrão Estrutural

Objetos de transferência de dados que separa a entrada (request) e a saída (response)

Mappers - Padrão de Conversão

Centralizam a conversão entre o modelo de domínio e os DTOs, evitando acoplamento

Controllers - Padrão MVC (camada)

Recebem as requisições HTTP, delegam ao serviço via DTOs e retornam a resposta

Builder - Padrão Criacional

Separa a construção de objetos complexos da sua representação. **Permite criar o mesmo tipo de objeto com diferentes configurações** de forma legível e segura.

Por que usamos no Dendê?

- Evento tem muitos parâmetros no construtor
- Evita valores nulos e construtores confuso

Builder - Padrão Criacional

```
// Difícil de manter: ordem dos campos é crítica e confusa
return new Evento(
    dto.nome(), dto.descricao(), dto.paginaEvento(),
    dto.dataInicio(), dto.dataFinal(), dto.tipo(),
    dto.modalidade(), dto.capacidadeMaxima(),
    dto.localAcesso(), dto.precoIngresso(),
    dto.permiteEstorno(), dto.taxaEstorno(),
    eventoPrincipal
);
```

Builder - Padrão Criacional

```
// Implementado no EventoMapper: claro e organizado
return new EventoBuilder()
    .nome(dto.nome())
    .descricao(dto.descricao())
    .dataInicio(dto.dataInicio())
    .precoIngresso(dto.precoIngresso())
    .eventoPrincipal(eventoPrincipal)
    .build();
```

DTO - Padrão Estrutural

DTO - Data Transfer Object

Por que usar?

- Protege o modelo de domínio, **expondo apenas o necessário e garantindo segurança** e desacoplamento entre API e domínio.

```java
public record AlterarPerfilComumDTO(
        String nome,
        LocalDate dataNascimento,
        Sexo sexo,
        String senha
) {}
```

```java
public abstract class Usuario {

    private String email;
    private String nome;
    private LocalDate dataNascimento;
    private Sexo sexo;
    private String senha;
    private Boolean ativo;
```

```java
    @PutMapping(path = "/{email}")
    public ResponseEntity<String> alterarUsuario(
            @PathVariable(parameter = "email") String email,
            @RequestBody AlterarPerfilComumDTO dto) {
        UsuarioComum usuario = repositorio.buscarUsuarioComum(email);
        if (usuario == null) return ResponseEntity.status(404,
            "Usuario nao encontrado.");

            usuario.alterarPerfil(dto);
            return ResponseEntity.ok("Perfil de " + email +
                " atualizado com sucesso.");
    }
```

DTO - Padrão Estrutural

- Exemplo de uso Request

```java
public static UsuarioComum toModel(CadastrarUsuarioComumRequestDto dto) {
    return new UsuarioComum(
        dto.nome(),
        dto.dataNascimento(),
        dto.sexo(),
        dto.email(),
        dto.senha()
    );
}
```

```java
public record CadastrarUsuarioComumRequestDto(
        String nome,
        LocalDate dataNascimento,
        Sexo sexo,
        String email,
        String senha
) {}
```

DTO - Padrão Estrutural

- Exemplo de uso Response

```java
public class Evento {
    private long id;
    private String nome;
    private String descricao;
    private String paginaEvento;
    private LocalDateTime dataInicio;
    private LocalDateTime dataFinal;
    private TipoEvento tipo;
    private ModalidadeEvento modalidade;
    private Integer capacidadeMaxima;
    private String localAcesso;
    private StatusEvento status;
    private BigDecimal precoIngresso;
    private Boolean permiteEstorno;
    private BigDecimal taxaEstorno;
    private Evento eventoPrincipal;
    private UsuarioOrganizador usuarioOrganizador;
```

```java
public record EventoResponseDTO(
    String nome,
    String descricao,
    LocalDateTime dataHoraInicio,
    LocalDateTime dataHoraFim,
    String nomeOrganizador,
    TipoEvento tipo,
    ModalidadeEvento modalidade,
    Integer vagasDisponiveis,
    String localAcesso,
    BigDecimal precoIngresso
){}
```

```java
@GetMapping
public ResponseEntity<List<EventoResponseDTO>> feedEventos() {
    List<EventoResponseDTO> lista = repositorio.feedEventos()
            .stream()
            .map(EventoMapper::toResponse)
            .toList();

    return ResponseEntity.ok(lista);
}
```

Mappers - Conversão

unex

O problema (sem Mapper)

Código "sujo": As Controllers ficam cheias de repetições para copiar dados de um objeto para outro.

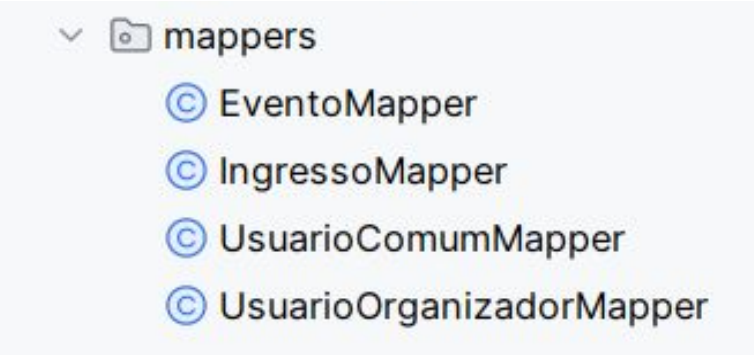
A solução (com Mapper)

Tradução Oficial: Centraliza a lógica de conversão entre o modelo de domínio e os DTOs em um único lugar.

Mappers - Conversão

Mappers criados: Usuário, Organizador, Evento, Ingresso

- toModel: converte request em domínio
- toResponse: converte domínio em response



Padrões de Projeto - Mappers

- Exemplo mapper Organizador - Centraliza a lógica de conversão.

```java
public static PerfilOrganizadorResponseDTO toResponse(UsuarioOrganizador organizador) {
    // Extrai dados da Empresa apenas se ela existir no domínio
    String nomeFantasia = (organizador.getEmpresa() != null) ?
                          organizador.getEmpresa().getNomeFantasia() : null;

    return new PerfilOrganizadorResponseDTO(
        organizador.getEmail(),
        organizador.getNome(),
        organizador.calcularIdade(),
        // ... outros campos
    );

}
```

Padrões de Projeto - Mappers

- Exemplo mapper Organizador - o controller delega a montagem da resposta ao Mapper

```
@GetMapping(path = "/{email}")
public ResponseEntity<?> visualizarPerfil(@PathVariable String email) {
    // Busca a entidade no repositório
    UsuarioOrganizador organizador = repositorio.buscarOrganizador(email);

    // Converte e retorna em uma única linha
    return ResponseEntity.ok(UsuarioOrganizadorMapper.toResponse(organizador));
}
```

Controllers - Camada

- **Objetivo:** Atuar como o ponto de contato entre o cliente e o sistema.
- **O que contém:** Endpoints (rotas), mapeamento de métodos HTTP (GET, POST, etc.) e validações básicas de entrada.

Responsabilidades:

- Receber e interpretar requisições HTTP.
- Validar dados iniciais e converter DTOs.
- Delegar a execução para a camada de Service
- Formatar e retornar a resposta final (Status Code e corpo).

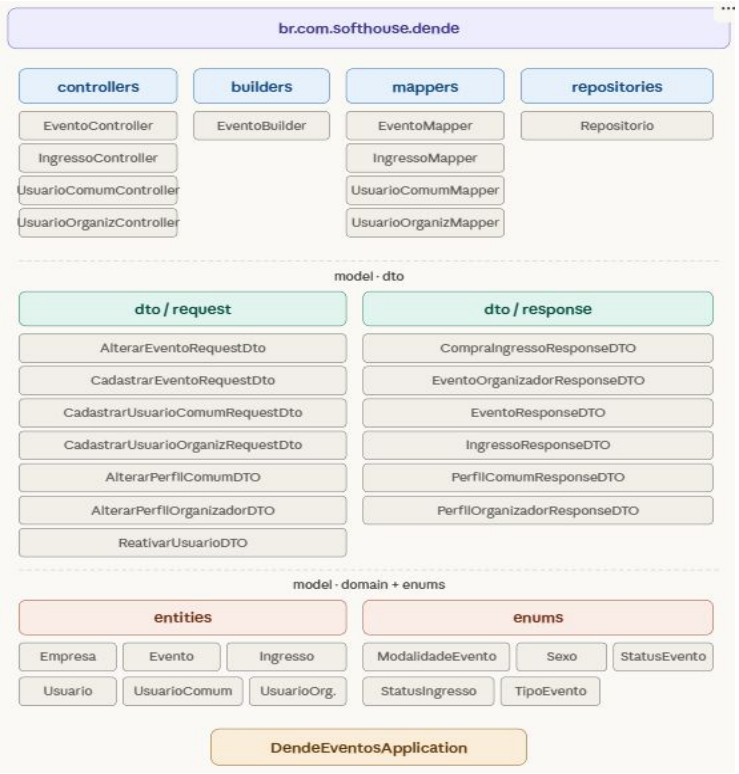
Não deve conter lógica de negócio; seu papel é puramente de orquestração de entrada e saída.

Padrões de Projeto - Controllers

- Exemplo: EventoController

```java
@RequestMapping(path = "/eventos")
public class EventoController {

    private final Repositorio repositorio;

    public EventoController() {
        this.repositorio = Repositorio.getInstance();
    }

    @GetMapping
    public ResponseEntity<List<EventoResponseDTO>> feedEventos() {List<EventoResponseDTO> lista = repositorio.feedEventos()
                .stream()
                .map(EventoMapper::toResponse)
                .toList();

        return ResponseEntity.ok(lista);
    }
}
```

Organização do Projeto



- **Separação clara de responsabilidades:** Cada pacote possui uma função específica (Builders, Controllers, Mappers, Models, Repositories).

- **Facilidade de manutenção e evolução:** A organização modular permite que alterações em uma camada não impactem diretamente as outras.

Considerações finais

- Separação de camadas mais adequada
- Código mais legível e manutenível
- Padrões aplicados de forma consistente

Referências

- ORACLE. **Java Platform, Standard Edition 17: API Specification.** [Redwood Shores]: Oracle, 2021. Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/index.html. Acesso em: 20 fev. 2026.